ECE 220 Computer Systems & Programming

Lecture 20 — Intro to C++: Objects, Constructors

November 3, 2026

Prof. Sayan Mitra mitras@illinois.edu
Course website: courses.grainger.illinois.edu/ece220/fa2026

ILLINOIS
Electrical & Computer Engineering
GRAINGER COLLEGE OF ENGINEERING

Adding Two Vectors: `vector.c`

```c
#include <stdio.h>
#include <math.h>

typedef struct VectorStruct {
    double angle, length;
} vector;

void scaleVector(vector *v, double k);
vector addVectors(vector a, vector b);

int main() {
    vector a = {1.5, 2}, b = {2.6, 3}, c;
    c = addVectors(a, b);
    scaleVector(&c, 2);
    printf("%f %f\n", c.length, c.angle);
    return 0;
}
```

The data is in the `struct`; the functions that act on it are declared beside it, and any code may reach into `c.length`.

Why Objects?

A `struct` says what a `vector` is made of.

A `class` says what you are allowed to do with it.

Why Objects?

A `struct` says what a `vector` is made of.
A `class` says what you are allowed to do with it.

- `scale`, `add`, `print` — the operations that make sense on a vector belong *to* the vector

Why Objects?

A `struct` says what a `vector` is made of.
A `class` says what you are allowed to do with it.

- `scale`, `add`, `print` — the operations that make sense on a vector belong *to* the vector

`scaleVector(&c, 2)` → `c.scale(2)`: **the vector carries its own verbs.**

Adding Two Vectors: `vector.c` and `vector.cpp`

```c
#include <stdio.h>
#include <math.h>

typedef struct VectorStruct {
    double angle, length;
} vector;

void scaleVector(vector *v, double k);
vector addVectors(vector a, vector b);

int main() {
    vector a = {1.5, 2}, b = {2.6, 3}, c;
    c = addVectors(a, b);
    scaleVector(&c, 2);
    printf("%f %f\n", c.length, c.angle);
    return 0;
}
```

Adding Two Vectors: `vector.c` and `vector.cpp`

```c
#include <stdio.h>
#include <math.h>

typedef struct VectorStruct {
    double angle, length;
} vector;

void scaleVector(vector *v, double k);
vector addVectors(vector a, vector b);

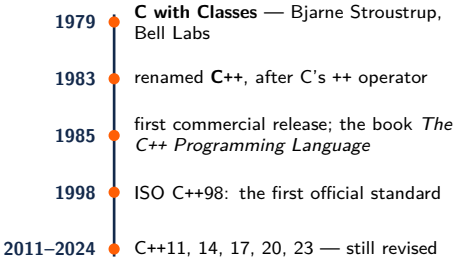
int main() {
    vector a = {1.5, 2}, b = {2.6, 3}, c;
    c = addVectors(a, b);
    scaleVector(&c, 2);
    printf("%f %f\n", c.length, c.angle);
    return 0;
}
```

```cpp
#include <iostream>
#include <cmath>

class vector {
    double angle_, length_;
  public:
    vector();
    vector(double a, double l);
    double getAngle() { return angle_; }
    double getLength() { return length_; }
    void scale(double k) { length_ *= k; }
    vector add(vector b);
};

int main() {
    vector a(1.5, 2), b(2.6, 3), c;
    c = a.add(b);
    c.scale(2);
    std::cout << c.getLength() << " "
              << c.getAngle() << std::endl;
    return 0;
}
```

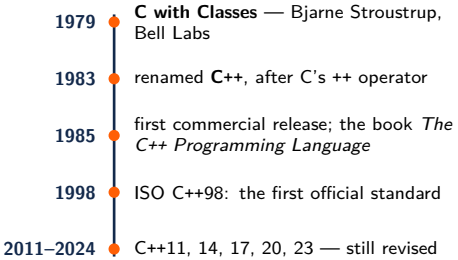
Where C++ Came From



C++ in the wild

- **GCC** and **Clang** — compilers you run
- **Chrome**, **Firefox** — browsers
- **TensorFlow**, **PyTorch** — the core of ML
- **Ingenuity**, the Mars helicopter — flight software

Where C++ Came From



C++ in the wild

- **GCC** and **Clang** — compilers you run
- **Chrome**, **Firefox** — browsers
- **TensorFlow**, **PyTorch** — the core of ML
- **Ingenuity**, the Mars helicopter — flight software

What it added to C

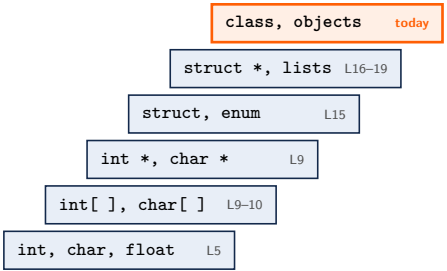
- Stroustrup wanted the program organization of Simula's classes at the speed of C
- **Classes**: data and the functions that work on it, in one unit with controlled access — *encapsulation*, today
- **Inheritance** and **polymorphism** (Lecture 21), **templates** (Lecture 22)
- Design rule: what you don't use, you don't pay for. A class like `vector` costs what the struct costs
- C stays inside: `vector.c` compiles unchanged as C++

Dates: B. Stroustrup, A History of C++: 1979–1991, HOPL-II, 1993; ISO/IEC 14882:1998. Design rule: B. Stroustrup, *The Design and Evolution of C++*, 1994.

Today's Three Questions

A class is a struct whose members are private by default and which includes functions.

1. What does `class` add to `struct`, and where does an object live?
2. Who initializes an object, and when does that code run?
3. What replaces `malloc` and `free`, and how can one name have several definitions?



The first of four C++ lectures: objects today, inheritance and polymorphism in Lecture 21, iterators and templates in Lecture 22, examples in Lecture 24.

Spot the Differences

	`vector.c`	`vector.cpp`
compiled with	`gcc`	_____
headers	`<stdio.h>`, `<math.h>`	_____
the type is a	`struct`: data	_____
`length` is open to	every function	_____
creating `a`	`vector a = {1.5, 2};`	_____
adding	`c = addVectors(a, b);`	_____
scaling	`scaleVector(&c, 2);`	_____
printing	`printf("%f", c.length);`	_____
it prints	`8.589099 2.172012`	`8.5891 2.17201`

Input and Output: `cin`, `cout`, Namespaces

```
#include <iostream>
using namespace std;

int main() {
    char name[20];
    cout << "Enter your name: ";
    cin >> name;          // like scanf("%s")
    cout << "Your name is: " << name << endl;
    return 0;
}
```

Rewrite `printf("%d %f\n", n, x);`

- `std::cout`: standard output stream; `<<` sends a value to it
- `std::cin`: standard input stream; `>>` reads into a variable
- `std::endl`: inserts a new-line and flushes the stream
- No format string and no `&`: the conversion comes from the _____
- A **namespace** is a named scope for identifiers. `std::cout` is the `cout` of namespace `std`
- `using namespace std;` lets the file write `cout` for `std::cout`

The library has its own `std::vector` (Lecture 22). With `using namespace std;` in `vector.cpp` the compiler reports *reference to 'vector' is ambiguous*, so that file writes `std::cout`.

From `struct` to `class`

```
class vector {
    double angle_, length_;
  public:
    vector();
    vector(double a, double l);
    double getAngle() { return angle_; }
    double getLength() { return length_; }
    void scale(double k) { length_ *= k; }
    vector add(vector b);
};
```

Add `c.length_ = 1;` to `main`:

_____

C++ keeps `struct`: a class whose members are public by default. Lecture 15's `student` is one.

- A **class** is a struct whose members are _____ by default and which carries _____
- An **object** is an instance of a class. `a`, `b`, `c` are three objects
- Each object has its own _____. All objects of a class share the _____
- A **member function** (method) is part of the class and is called on an object: `a.add(b)`
- `private` members are open only to _____, `public` members to anyone
- `main` reads `length_` by calling _____

Data and the functions that work on it in one unit, with controlled access: **encapsulation**.

Where an Object Lives, and What `this` Is

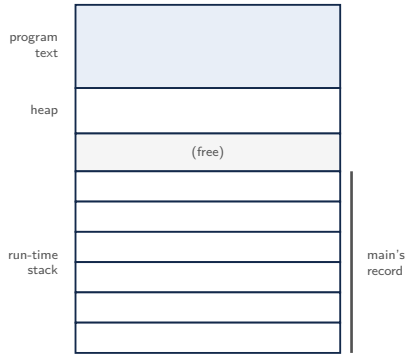
```
void scaleVector(vector *v, double k) {
    v->length *= k;
}
    scaleVector(&c, 2);
```

```
    void scale(double k) {
        std::cout << "this = " << this << std::endl;
        this->length_ *= k;
    }
    std::cout << "size = " << sizeof(c) << std::endl;
    std::cout << "&c   = " << &c << std::endl;
    c.scale(2);
```

`c.scale(2)` passes a hidden first argument: _____

One run of `this.cpp` printed `size =` _____

and `&c = 0x16cfca408`, `this =` _____



Fill in the functions, and `a`, `b`, `c` at the end of `vector.cpp`'s `main`.

Who gives an object its first values? Constructors, next.

In-Class Problem: Rectangle's Constructors

A **constructor** is a member function invoked automatically to _____ a new object and _____ its data members.

```
class Rectangle {
    int width_, height_;
  public:
    Rectangle();
    Rectangle(int, int);
    int area() const { return width_ * height_; }
};
```

- Its name is the class name. It has no return type
- `Rectangle::` says which class the function belongs to
- `area() const` does not change the object
- `rect1(3, 4)` runs _____
- `Rectangle rect2;` runs _____

```
Rectangle::Rectangle() {
    // set both width_ and height_ to 0




}

Rectangle::Rectangle(int w, int h) {
    // set width_ to w and height_ to h




}
```

When Constructors and Destructors Run

A **destructor** is invoked automatically to _____ an object when it goes out of scope or is explicitly deleted.

```
Rectangle::~Rectangle() {
    cout << "destruct " << width_ << "x"
         << height_ << endl;
}
```

```
int main() {
    Rectangle rect1(3, 4);
    cout << "rect1 area " << rect1.area() << endl;
    {
        Rectangle rect2;
        cout << "rect2 area " << rect2.area() << endl;
    }
    cout << "end of main" << endl;
    return 0;
}
```

Each constructor in `lifetime.cpp` prints a line too. The program prints:

```
_____

rect1 area _____

_____

rect2 area _____

_____

end of main

_____
```

`rect1` and `rect2` live on the run-time stack. Objects on the heap, next.

new and delete

	C (Lecture 16)	**C++**
one `int`	`p = (int *)malloc(sizeof(int));`	_____
release it	`free(p);`	_____
array of 10	`q = (int *)malloc(10 * sizeof(int));`	_____
release it	`free(q);`	_____
they are	library functions from `<stdlib.h>`	_____
size and type	you compute the size and cast the `void *`	_____
for an object	only bytes: no constructor runs	run the _____ and _____

- The heap and its rules are those of Lecture 16: memory from `new` lives until `delete`, and a missing `delete` is a _____
- `new` pairs with `delete`, and `new[]` with `delete[]`

In-Class Problem: Objects Through Pointers

```
int main() {
    Rectangle rect1(3, 4);
    Rectangle *r_ptr1 = &rect1;
    // print rect1's area through r_ptr1


    Rectangle *r_ptr2 = new Rectangle(5, 6);
    // print the area of *r_ptr2


    Rectangle *r_ptr3 = new Rectangle[2]{
        Rectangle(), Rectangle(2, 4)};
    // print the areas of both rectangles


    // deallocate the heap memory


    return 0;
}
```

- `main` prints _____, _____, _____, _____
- Objects on the heap: _____. On the stack: _____

Function Overloading

```
int getmin(int a, int b) {
    return (a < b) ? a : b;
}

double getmin(double a, double b) {
    return (a < b) ? a : b;
}

int main() {
    cout << getmin(3, 5) << endl;
    cout << getmin(2.5, 1.5) << endl;
    return 0;
}
```

- In C a function name has exactly one definition
- C++ allows **overloading**: several functions with one name and different _____
- At each call the compiler chooses by the _____
- `getmin(3, 5)` runs the _____ version
- `getmin(2.5, 1.5)` runs the _____ version
- The program prints _____ and _____
- `getmin(3, 2.5)`: _____
- Overloaded earlier today: _____

The two bodies are the same text. Lecture 22 writes it once, as a template.

Operator Overloading

```
class vector {
  protected:
    double angle_, length_;
  public:
    vector operator+(const vector &b) {
        vector c;
        double ax = length_ * cos(angle_);
        double bx = b.length_ * cos(b.angle_);
        double ay = length_ * sin(angle_);
        double by = b.length_ * sin(b.angle_);
        double cx = ax + bx;
        double cy = ay + by;
        c.length_ = sqrt(cx * cx + cy * cy);
        c.angle_ = acos(cx / c.length_);
        return c;
    }
    // constructors and the other member functions
```

```
int main() {
    vector a(1.5, 2), b(2.6, 3);
    vector c = a + b;          // before: a.add(b)
    std::cout << c.getLength() << " "
              << c.getAngle() << std::endl;
    return 0;
}
```

- A class can redefine the built-in operators for its objects: `+ - * < > == =`
- `a + b` is the call _____
- Inside it `this` points to _____
- `const vector &b` is a reference (Lecture 21): `main`'s `b` is not copied, and `operator+` may not change it
- `protected`: private, and open to derived classes (Lecture 21)

`a + b` is the opening program's `a.add(b)` under a new name. It prints `4.29455 2.17201`.

Summary & What's Next

Today

- **C vs C++**: `g++`, `<iostream>`, `cout <<` and `cin >>`. The library's names live in namespace `std`
- **Class**: a struct with private members by default and member functions. An object is laid out like a struct, and a member function receives its address in `this`
- **Constructors** run when an object is created. The **destructor** runs at the end of its scope or at `delete`
- **`new`, `delete`, `delete[]`** replace `malloc` and `free`, and run constructors and destructors
- **Overloading**: one name, several parameter lists (`getmin`). Operators are functions too (`operator+`)

Midterm 2 is Thursday, November 5. No lecture that day.

Next lecture (Tuesday, November 10)

- Inheritance and polymorphism: references, the copy constructor, a base class `Shape` with derived `Rectangle` and `Triangle`, `virtual`

Code from today: `vector.c`, `vector.cpp`, `io.cpp`, `this.cpp`, `rect.cpp`, `lifetime.cpp`, `alloc.c`, `alloc.cpp`, `rect_ptr.cpp`, `getmin.cpp`, `vector_plus.cpp`.